Generating Novel Origami Crease Patterns Using Graph Neural Networks, Genetic Algorithms, and Origami Constraints

Arjun Singh, Yuvraj Verma

May 3, 2026

Abstract

This project explores the generation of novel origami crease patterns using graph theory, graph neural networks, genetic algorithms, and mathematical constraint repair. We represent each crease pattern as a planar graph, where vertices store geometric information and edges represent border, mountain, or valley folds. A vertex-based graph neural network and a line-graph neural network are used to learn validity from real and corrupted crease-pattern data. These learned models guide a genetic algorithm that evolves new crease-pattern candidates from randomly generated planar graph seeds.

Since visual similarity to origami is not enough to guarantee mathematical validity, the generator also applies explicit repair methods based on origami constraints. The final method uses topology repair, Z3 Maekawa repair, Kawasaki projection repair, gradient geometry repair, symmetry enforcement, crossing checks, novelty penalties, and hard-gated fitness evaluation. The best generated candidates are crossing-free, Maekawa-valid, Z3-satisfiable, and show substantially reduced Kawasaki violation compared to earlier versions. The project demonstrates a hybrid method for generating locally plausible and mathematically constrained origami crease-pattern candidates.

Contents

1 Introduction **4**

2 Problem Statement **5**

3 Our Contribution **6**

3.1 Graph-Based Representation of Crease Patterns 6

3.2 Learned Validity Using Graph Neural Networks 6
3.3 Symmetry-Constrained Genetic Algorithm 7
3.4 Constraint Repair 7
3.5 A Thorough Fitness Function 7

4 Literature Survey and Pre-requisites **8**
4.1 Crease 8
4.2 Crease Pattern 8
4.3 Flat Origami 8
4.4 Single Vertex Flat-Foldability 9
4.4.1 Kawasaki's Theorem 9
4.4.2 Maekawa's Theorem 10
4.4.3 Big-Little-Big Theorem 10
4.4.4 Two Colorability Theorem: Kawasaki's Second Theorem 11
4.5 Proof of Maekawa's Theorem 12
4.6 Proof of Kawasaki's Theorem 14

5 Algorithm Built **16**
5.1 Pseudocode 16
5.2 Explanation 17

6 Methodology, Algorithms, and Data Structures Used **18**
6.1 Crease-Pattern Graph Representation 18
6.2 Vertex-Based Graph Neural Network 19
6.3 Line-Graph Classifier 19
6.4 Genetic Algorithm 20
6.5 Topology Repair 20
6.6 Z3 Maekawa Repair 20
6.7 Kawasaki Projection and Gradient Geometry Repair 21

6.8 Symmetry Handling 21
6.9 Fitness Function 22
6.10 Overall Repair Sequence 22

7 Highlights of Coding and Implementation Details 23

8 Case Studies and Results 24

8.1 Baseline Genetic Algorithm 24
8.2 Line-GNN Filtering 24
8.3 Z3 and Topology Repair 24
8.4 Kawasaki-Focused Repair 25
8.5 Symmetry-Constrained Generation 26
8.6 Final Results: 27

9 Bonus: Dashboard, Dataset, and Reusable Code 30

10 Conclusions and Possible Future Directions 32

1 Introduction

Origami is the art of folding paper. Traditionally, it involves folding a single square sheet into various objects, animals, shapes, structures, or patterns. A lot of what motivates and fascinates origami enthusiasts is the fact that something as simple as a piece of paper can transform into something beautiful just by folding it. Origami has evolved significantly since its origins, with artists now creating highly intricate and even lifelike models from just one square of paper.

The mathematics behind origami has advanced alongside the increasing complexity of these models. It has been successful in solving problems that were unsolvable using classical geometry, such as the trisection of an angle, solving cubic equations, and other geometric constructions. Origami also has applications in astronomy, medicine, design, robotics, engineering, and more.

Simultaneously, the field of origami design and folding has seen major developments through computational methods. Examples include Robert Lang's TreeMaker, Erik Demaine's Fold-and-Cut Theorem, Mu-Tsun Tsai's BoxPleat Studio, Orihime by Toshiyuki Meguro, and many others.

Our project focuses on generating novel origami crease patterns using a combination of graph theory, graph neural networks, genetic algorithms, and mathematical constraint repair. We represent each crease pattern as a NetworkX graph. In this representation, nodes store geometric information such as coordinates, degree, and angular features, while edges store fold labels. The fold labels distinguish between border edges, mountain folds, and valley folds. This graph representation allows us to treat crease-pattern generation as a structured graph-generation problem rather than only as a drawing or image-generation task.

The first stage of the project involves building a dataset of valid crease-pattern graphs from real `.cp` files. These files are parsed into graph objects and used to train graph neural network classifiers that learn what valid origami crease patterns look like. The main classifier is a vertex-based Graph Isomorphism Network, which takes node features and graph connectivity as input and predicts whether a crease-pattern graph is valid or invalid. In addition to this, we also use a line-graph classifier, where creases are treated as graph nodes. This gives an additional crease-level validity signal and helps filter out patterns that may look plausible at the vertex level but are structurally weak at the crease level.

After training the classifiers, we use a genetic algorithm to generate new crease-pattern candidates. The genetic algorithm begins with randomly generated planar graphs and evolves them through mutation and selection. Each candidate is scored using several criteria, including GNN validity confidence, Line-GNN confidence, novelty, complexity, symmetry, Kawasaki penalty, and Maekawa penalty. The purpose of the genetic algorithm is not simply to reproduce existing crease patterns, but to search for new graph structures that are both novel and close to satisfying origami validity constraints.

However, learned validity alone is not enough to guarantee mathematically meaningful

origami crease patterns. Therefore, the pipeline also includes explicit constraint-repair methods. An even-degree topology repair step modifies graphs so that interior non-border vertices are more suitable for Maekawa repair. A Z3-based Maekawa repair step then assigns or repairs mountain and valley labels so that interior vertices satisfy Maekawa's theorem. To improve Kawasaki's theorem, the system uses a direct Kawasaki projection repair, which adjusts the sector angles around problematic vertices so that alternating angle sums move closer to π. Finally, a gradient-based geometry repair further adjusts mutable node coordinates to reduce Kawasaki residuals, improve symmetry, avoid short-edge degeneracy, and reject repairs that create edge crossings.

Overall, this project develops a hybrid generation pipeline for origami crease patterns. The graph neural networks provide learned validity signals, while the genetic algorithm explores the space of possible crease-pattern graphs. The symbolic and geometric repair modules then enforce important origami constraints. The goal is not merely to generate graphs that visually resemble origami, but to move toward crease-pattern candidates that are novel, crossing-free, locally foldable, and mathematically consistent with Maekawa's and Kawasaki's theorems.

2 Problem Statement

The central problem of this project is to generate novel origami crease-pattern candidates that satisfy important mathematical and structural properties of flat-foldable origami.

A crease pattern is a planar graph embedded on a sheet of paper, where the edges represent creases and the vertices represent intersections of creases. Each crease must also have a fold assignment: border, mountain, or valley. Therefore, generating a crease pattern is more difficult than generating an arbitrary planar graph. A candidate pattern must satisfy geometric, combinatorial, and physical constraints at the same time.

We want to generate graphs that satisfy the following requirements:

- The graph should be embedded inside a square paper boundary.
- The graph should be planar, meaning crease edges should not cross except at shared vertices.
- The graph should contain meaningful mountain, valley, and border labels.
- Interior vertices should satisfy Maekawa's theorem, so that the number of mountain and valley folds is locally consistent with flat-foldability.
- Sector angles around interior vertices should approximately satisfy Kawasaki's theorem.
- The generated pattern should be novel, rather than a direct copy of a real crease pattern from the dataset.
- The generated pattern should have enough structural complexity to be interesting.

This problem is tough because these requirements interact with each other. For example, moving a vertex may reduce Kawasaki violation at one point but create edge crossings elsewhere. Similarly, changing a mountain fold to a valley fold may improve Maekawa's theorem at one vertex but break it at a neighboring vertex. A graph may also look visually similar to a crease pattern while still failing the mathematical constraints required for local flat-foldability.

The main research question of this project is then: **Can we build a pipeline that learns structural features of valid origami crease patterns and use it with graph-theoretic and geometric constraint repair to generate new locally valid crease-patterns?**

3 Our Contribution

We build a hybrid pipeline for generating origami crease-pattern candidates using graph neural networks, genetic algorithms, and explicit origami constraint repair.

3.1 Graph-Based Representation of Crease Patterns

We add crease patterns as graph objects. Each crease pattern is stored as a NetworkX graph, where nodes contain geometric information such as coordinates, degree, and angle features, while edges contain fold-type labels. In this representation, each edge has a fold label

$$\texttt{fold_type} \in \{1, 2, 3\},$$

where 1 denotes a border edge, 2 denotes a mountain fold, and 3 denotes a valley fold. This representation allows origami crease-pattern generation to be treated as a graph-generation problem. It also makes it possible to compute local origami constraints such as Kawasaki violation, Maekawa violation, planarity, and symmetry directly from the graph.

3.2 Learned Validity Using Graph Neural Networks

A vertex-based Graph Isomorphism Network is trained on real and corrupted crease-pattern graphs. This model learns to classify whether a graph resembles a valid crease pattern.

In addition to the vertex-based GNN, the project also uses a line-graph GNN. In the line-graph representation, creases are treated as graph nodes. This gives us crease-level validity that complements the vertex-level classifier. Together, these two models provide learned guidance for the generator. This learned scoring helps the genetic algorithm search more intelligently than random mutation alone.

3.3 Symmetry-Constrained Genetic Algorithm

We build a symmetry-aware genetic algorithm for crease-pattern generation. The genetic algorithm begins with random planar graph seeds inside a square boundary. It then evolves candidate graphs using mutation, selection, repair, and fitness evaluation.

The final version of the algorithm can enforce either vertical or diagonal symmetry. When symmetry is enabled, the algorithm extracts one half of the graph, mutates and repairs that half, and then reflects it to reconstruct the full crease pattern. This reduces the search space and encourages cleaner, more origami-like structures. The algorithm also periodically injects repaired random seeds into the population to prevent the search from collapsing too quickly into one local family of designs.

3.4 Constraint Repair

We integrate origami repair modules into the genetic algorithm. Since learned validity scores alone cannot guarantee mathematical correctness, each generated candidate is passed through a sequence of repair steps.

The repair pipeline consists of tolpology repair, Z3 Maekawa Repair, Kawasaki Projection Repair, Gradient Geometry Repair.

The topology repair step attempts to fix odd-degree interior vertices so that the graph is more suitable for Maekawa repair. The Z3 Maekawa repair step treats mountain-valley assignments as discrete variables and solves for assignments that satisfy Maekawa's theorem. The Kawasaki projection repair step directly adjusts sector angles around high-violation vertices so that alternating angle sums move closer to π. Finally, the gradient geometry repair step fine-tunes node coordinates while preserving the boundary and avoiding edge crossings.

3.5 A Thorough Fitness Function

Earlier versions of the generator allowed invalid candidates to survive if they received a high GNN score. Our version prevents this by rejecting candidates that fail validity requirements.

A candidate is assigned a very low fitness score if it has illegal edge crossings, fails topology repair, fails Z3 Maekawa repair, or has Kawasaki violation above a certain threshold.

Only candidates that pass these requirements are scored using the full fitness function, which combines GNN and line-GNN confidence, complexity, novelty, symmetry, and remaining constraint penalties.

A simplified version of the fitness function is:

$$F(G) = P_{\text{GNN}}(G) + \lambda_C C(G) - \lambda_K K(G) - \lambda_M M(G) - \lambda_S S(G) - \lambda_N N(G) - L(G),$$

where $P_{\text{GNN}}(G)$ is the vertex-GNN validity score, $C(G)$ is the complexity bonus, $K(G)$ is the Kawasaki penalty, $M(G)$ is the Maekawa penalty, $S(G)$ is the symmetry penalty, $N(G)$ is the novelty penalty, and $L(G)$ is the line-GNN penalty.

The Kawasaki gate becomes stricter over generations. This allows broader exploration early in the run and stronger geometric validity later in the run.

4 Literature Survey and Pre-requisites

4.1 Crease

A crease is defined as a line segment which can take upon one of two assignments: mountain or valley. A mountain fold is folded away from the viewer, while a valley fold is folded toward the viewer. These assignments describe the orientation with which the crease must be folded. In this paper, a red line refers to a mountain fold while a blue line refers to a valley fold.

4.2 Crease Pattern

A crease pattern is defined as a finite planar straight-line graph, where the edges are creases and the vertices are points of intersection of the creases. Curved folds are not considered here because the project focuses on the straight-line crease-pattern setting used for flat-foldability constraints.

4.3 Flat Origami

Consider a set of creases C forming a crease pattern lying on a convex planar region $P \subset \mathbb{R}^2$. We define C to be flat-foldable if there exists a function

$$f : P \to \mathbb{R}^2$$

such that:

- f is an isometric mapping, meaning it preserves distances within each face of the crease pattern.
- f is differentiable everywhere on P, except along the creases in C, where folding introduces sharp turns.
- The image $f(P)$ lies entirely in the plane $\mathbb{R}^2$, i.e., the folded result is flat.

4.4 Single Vertex Flat-Foldability

For a single vertex to be flat-foldable in a given mountain-valley assigned crease pattern, there exist three important local conditions: Kawasaki's theorem, Maekawa's theorem, and the Big-Little-Big condition. These conditions are central to this project because they give local mathematical tests for whether a generated crease-pattern graph is plausible as an origami crease pattern.

4.4.1 Kawasaki's Theorem

The alternating sum of angles around a vertex must equal 180°. If the angles between consecutive creases are labeled

$$\alpha_1, \alpha_2, \ldots, \alpha_{2n},$$

arranged in order around the vertex, then:

$$\alpha_1 + \alpha_3 + \cdots + \alpha_{2n-1} = \alpha_2 + \alpha_4 + \cdots + \alpha_{2n} = \pi.$$

Equivalently,

$$\alpha_1 - \alpha_2 + \alpha_3 - \cdots + \alpha_{2n} = 0.$$

Figure 1 is an example of a circular disk D which is flat-foldable without a mountain-valley assignment. This means that under this condition there exists a mountain-valley assignment such that D can be folded flat.

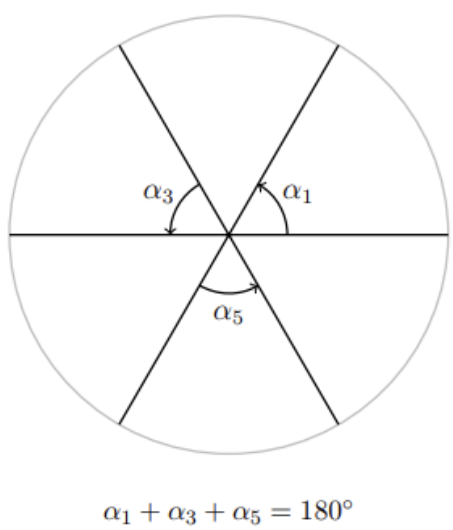

Figure 1: Example of a circular disk satisfying Kawasaki's theorem.

4.4.2 Maekawa's Theorem

For any vertex in a crease pattern, the difference between the number of mountain folds and valley folds incident to that vertex must be ± 2. That is,

$$|M| - |V| = \pm 2,$$

where $|M|$ is the number of mountain folds and $|V|$ is the number of valley folds at the vertex.

In this project, Maekawa's theorem is handled using a Z3 repair module. The module treats mountain-valley assignments as Boolean variables and attempts to repair fold labels so that interior vertices satisfy the theorem.

4.4.3 Big-Little-Big Theorem

If an angle α_i between two adjacent creases at a vertex is smaller than both its neighboring angles, meaning the angle is locally minimum, i.e.,

$$\alpha_i < \alpha_{i-1} \quad \text{and} \quad \alpha_i < \alpha_{i+1},$$

then the two creases forming this small angle must be assigned opposite mountain-valley fold types.

This condition ensures that no part of the paper attempts to pass through itself during folding. If two folds with the same assignment, both mountain or both valley, form the boundaries of a very narrow wedge, then during flattening the two adjacent layers of paper would attempt to occupy the same space, leading to an intersection. This makes the fold physically impossible. In Figure 2, both Kawasaki's and Maekawa's theorem are satisfied, yet the figure is not flat-foldable because the locally minimum angle on top has both valley assignments.

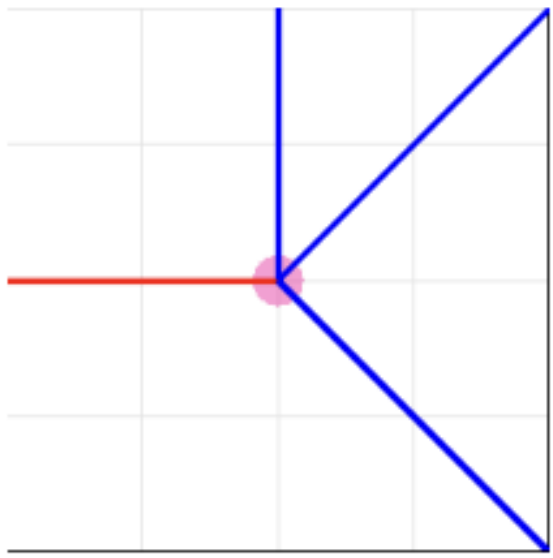

Figure 2: Example where Kawasaki's and Maekawa's theorems are satisfied, but the pattern is not flat-foldable because the Big-Little-Big condition fails.

4.4.4 Two Colorability Theorem: Kawasaki's Second Theorem

Given a flat-foldable origami model, unfolding it gives a crease pattern, which forms a planar graph embedded in the plane. The edges correspond to creases, and the regions enclosed by the creases form the faces of the planar embedding.

Kawasaki's second theorem states that the faces of the corresponding planar graph are 2-colorable. That is, it is possible to assign one of two colors to each face such that no two adjacent faces, meaning faces sharing a common edge, have the same color.

If G is the planar graph defined by the crease pattern, then its dual graph G^*, in which each face of G becomes a vertex of G^*, and an edge connects two vertices of G^* if the corresponding faces in G share a boundary, is bipartite. A bipartite dual graph implies that the faces of G can be properly colored using only two colors.

This 2-colorability ensures that the regions of any flat-foldable crease pattern can always be partitioned into two sets such that no two adjacent regions lie in the same set. Figure 3 shows a two-coloured crease pattern of the traditional crane.

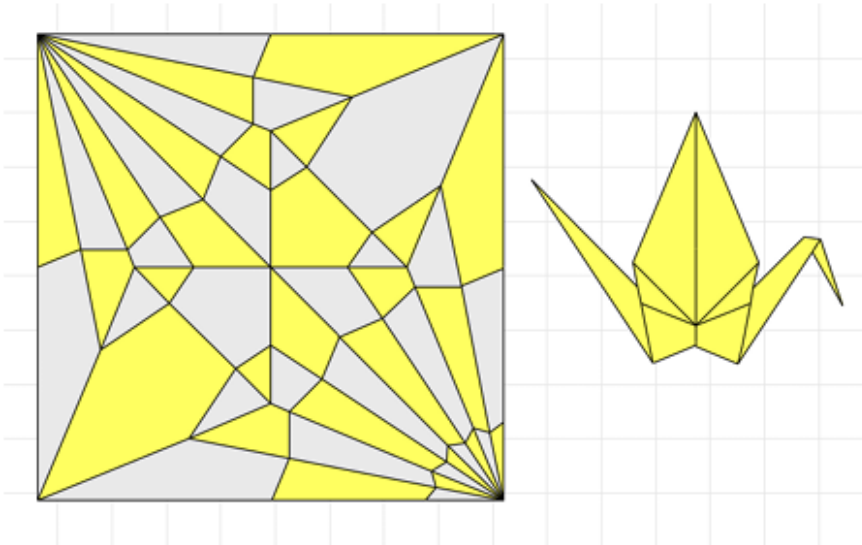

Figure 3: Two-coloured crease pattern of the traditional crane.

For a given mountain-valley assigned crease pattern, these conditions are necessary and sufficient for individual vertex flat-foldability. These conditions are necessary but not sufficient in the case of folding multiple vertices together, or global flat-foldability:

1. **Kawasaki's Theorem (K1):** Ensures the crease angles fold flat geometrically.
2. **Maekawa's Theorem (M):** Ensures the correct number of mountain versus valley folds.
3. **Big-Little-Big (K2):** Prevents local self-intersection by requiring alternating mountain-valley folds across narrow angles.

4.5 Proof of Maekawa's Theorem

Proof. Imagine that we take a small circular region around the vertex after folding and cut it horizontally, creating a polygonal cross-section of the paper. Since we are working with a flat-folded model, this cross-section lies flat in the plane and forms a simple polygon.

The internal angles at each fold in this polygon depend on whether the fold is a mountain fold or a valley fold. In one orientation, every mountain fold contributes an angle of 0, while every valley fold contributes an angle of 2π.

Let M be the number of mountain creases and V be the number of valley creases incident to the vertex. Since every crease in the pattern contributes a vertex to this polygon, the total number of sides of the polygon is

$$n = M + V.$$

The sum of the interior angles of any n-gon is

$$(n-2)\pi.$$

Therefore, in this orientation, we get

$$0 \cdot M + 2\pi \cdot V = (M + V - 2)\pi.$$

Simplifying,

$$2\pi V = (M + V - 2)\pi.$$

Dividing both sides by π, we get

$$2V = M + V - 2.$$

Rearranging gives

$$V = M - 2.$$

Hence,

$$M - V = 2.$$

This proof assumes that the vertex points "upward" after folding. If the vertex instead points "downward", then the roles of mountain and valley folds are reversed: valley folds correspond to angle 0 and mountain folds correspond to angle 2π. In that case, we get

$$V - M = 2.$$

Therefore, in general,

$$|M - V| = 2.$$

This proves Maekawa's theorem. $\square$

Corollary: The Number of Creases at a Flat-Foldable Vertex is Even

Proof. Let

$$n = M + V.$$

From Maekawa's theorem, we know that

$$M - V = \pm 2.$$

Now,

$$n = M + V.$$

We can rewrite this as

$$n = (M - V) + 2V.$$

Substituting Maekawa's condition gives

$$n = \pm 2 + 2V.$$

In both cases, n is even because it is the sum of two even numbers. Hence, the number of creases at a flat-foldable vertex must always be even. □

4.6 Proof of Kawasaki's Theorem

Proof. Label the $2n$ angles around the vertex as

$$\alpha_1, \alpha_2, \ldots, \alpha_{2n},$$

in the order in which they appear around the vertex. When we fold the paper along the creases, the paper must lie flat. This means that there is no net excess turn around the vertex when we travel around it.

Consider a small loop encircling the vertex. As this loop is traced on the surface of the folded model, it is redirected every time it crosses a crease. Since we end up back where we started, the total alternating change in direction is zero:

$$\alpha_1 - \alpha_2 + \alpha_3 - \cdots - \alpha_{2n} = 0.$$

Rearranging this equation gives

$$\alpha_1 + \alpha_3 + \cdots + \alpha_{2n-1} = \alpha_2 + \alpha_4 + \cdots + \alpha_{2n}.$$

Also, since the angles around the vertex form a full circle, we have

$$\alpha_1 + \alpha_2 + \cdots + \alpha_{2n} = 2\pi.$$

The odd-indexed angles and even-indexed angles have equal sums, and together they add to 2π. Therefore, each sum must be equal to π:

$$\alpha_1 + \alpha_3 + \cdots + \alpha_{2n-1} = \alpha_2 + \alpha_4 + \cdots + \alpha_{2n} = \pi.$$

Therefore, Kawasaki's theorem holds. □

Why Kawasaki's Condition is Necessary and Sufficient

The alternating angle condition is not only necessary for single-vertex flat-foldability, but also sufficient. This result, proved by Thomas Hull, tells us that for a single vertex, local flat-foldability depends on the angles between consecutive creases.

In other words, if the angles around a single vertex satisfy Kawasaki's theorem, then there exists at least one mountain-valley assignment that allows the vertex to fold flat. Therefore, for a single vertex, checking the alternating angle sums is enough to decide whether the vertex is locally flat-foldable.

This is important for our project because the Kawasaki penalty is computed from the sector angles around each interior vertex. Reducing this penalty means moving the generated crease pattern closer to satisfying the local geometric condition for flat-foldability.

Connection to Two-Colorability

From the corollary to Maekawa's theorem, the number of creases around a flat-foldable vertex is even. Therefore, around a flat-foldable vertex, we can color one face with one color and then alternate colors as we move around the vertex. Since the number of surrounding faces is even, this alternating coloring returns consistently to the starting face.

This gives the intuition behind Kawasaki's second theorem: the faces of a flat-foldable crease pattern are two-colorable. That is, the regions of the crease pattern can be colored using two colors so that adjacent faces receive different colors.

5 Algorithm Built

The final generator is a symmetry-constrained genetic algorithm. It starts from randomly generated planar crease-pattern graphs, repairs each candidate using origami constraints, and then evolves the population through selection, mutation, and learned validity scoring. The key point is that candidates are not selected only because they receive a high GNN score; they must also pass explicit checks for crossings, topology, Maekawa validity, and Kawasaki violation.

5.1 Pseudocode

Crease-Pattern Generation Algorithm

Require: Population size N, generations T, elite count E, mutations per child m
Require: Trained vertex-GNN, line-GNN, real crease-pattern dataset
Require: Symmetry setting: vertical, diagonal, or none

```
Initialize population P ← ∅
for i = 1 to N do
    Generate a random planar crease-pattern graph G
    if symmetry is enabled then
        Extract one half of G
        Reflect the half-graph to form a symmetric full graph
    end if
    Rebuild the square boundary of G
    Remove illegal crossing edges
    Apply repair pipeline to G
    Add repaired graph G to P
end for
for t = 1 to T do
    Score every graph G ∈ P using hard-gated fitness
    Store the best graph found so far
    Select the top E graphs as elites
    Compute shared fitness to discourage near-duplicate candidates
    Select parents from the stronger candidates
    Initialize new population P_new with the elites
    while |P_new| < N do
        Select a parent graph G
        Create a copy G' of the parent
```

```
        for j = 1 to m do
            Apply one mutation to G'    ▷ flip fold, move node, add edge, or remove edge
            if symmetry is enabled then
                Reflect the mutated half-graph to reconstruct the full graph
            end if
        end for
        Apply repair pipeline to G'
        Add repaired child G' to P_new
    end while
    if t is a diversity-injection generation then
        Generate and repair fresh random seeds
        Replace some weak candidates with these seeds
    end if
    if symmetry is enabled and t is a symmetry-repair generation then
        for each graph G ∈ P_new do
            Re-enforce symmetry
            Apply repair pipeline again
        end for
    end if
    P ← P_new
end for
Re-score the final population
Select the best graph and top diverse candidates
Save graphs, visualizations, heatmaps, and editable .cp files
```

5.2 Explanation

The initial population is created from scratch using random planar graph seeds inside a square boundary. If symmetry is enabled, the algorithm works on one half of the graph and reflects it to obtain the full crease pattern. This makes the search space smaller and encourages more structured outputs.

After initialization and after every mutation, the same repair pipeline is applied:

$$\begin{aligned}&\text{Topology Repair}\\&\rightarrow \text{Z3 Maekawa Repair}\\&\rightarrow \text{Kawasaki Projection Repair}\\&\rightarrow \text{Gradient Geometry Repair.}\end{aligned}$$

This repair step is necessary because mutation can easily break local origami constraints. The repaired candidates are then scored using a hard-gated fitness function. Candidates with crossings, failed topology repair, failed Z3 repair, nonzero Maekawa violation, or excessive Kawasaki violation are rejected before the learned GNN scores are considered.

The remaining candidates are ranked using GNN validity, line-GNN validity, novelty, complexity, symmetry, and remaining constraint penalties. The algorithm keeps elite candidates, uses shared fitness to avoid selecting too many near-duplicates, mutates selected parents, and occasionally injects fresh repaired seeds to maintain diversity. At the end, it exports the best candidates both as graph objects and as editable `.cp` crease-pattern files.

6 Methodology, Algorithms, and Data Structures Used

The final pipeline is implemented in `Genetic_Algo_Z3_Topology_Symmetry_Kawasaki.py`. It is a hybrid system that combines graph neural networks, a genetic algorithm, topology repair, symbolic constraint solving with Z3, direct Kawasaki projection, gradient-based geometry repair, and symmetry-constrained generation. The motivation for this hybrid design is that no single method was sufficient on its own.

A learned classifier can estimate whether a candidate resembles real crease patterns, but it cannot guarantee exact satisfaction of Maekawa's theorem or Kawasaki's theorem. A symbolic solver can repair mountain-valley labels, but it cannot move vertices to fix sector angles. A geometric optimizer can reduce Kawasaki residuals, but it cannot solve the discrete mountain-valley assignment problem. The final system therefore uses each method for the part of the problem it is naturally suited for.

6.1 Crease-Pattern Graph Representation

The basic object in the project is a crease pattern represented as a NetworkX graph

$$G = (V, E).$$

Each vertex stores geometric information, and each edge stores a crease assignment. In our convention, an edge label of 1 denotes the paper boundary, 2 denotes a mountain fold, and 3 denotes a valley fold.

Each node stores planar coordinates (x, y), its degree, and angular information about incident creases. These angular features are important because Kawasaki's theorem depends on the sector angles around each vertex. A candidate graph is therefore not treated as a purely combinatorial object; it is a straight-line embedded planar graph inside a square sheet.

The original dataset was built from real `.cp` crease-pattern files. Each line in a `.cp` file has the form

```
fold_type x1 y1 x2 y2
```

where `fold_type` is the crease label. The parser converts these line segments into graph edges, merges endpoints into graph vertices, computes local node features, and stores the

resulting NetworkX graphs in `graphs.pkl`. In the current dataset, 626 real crease-pattern graphs are used as the main positive reference set.

Negative examples are generated by corrupting real crease patterns in controlled ways, such as flipping mountain-valley labels, jittering or moving interior vertices, deleting edges, or adding random creases. These corruptions are useful because they force the classifier to learn origami-specific constraints instead of only recognizing that the graph visually contains lines.

6.2 Vertex-Based Graph Neural Network

The first classifier is a Graph Isomorphism Network. It operates directly on the crease-pattern graph. The architecture uses three GIN convolution layers with MLPs, batch normalization, ReLU activations, and global mean/max pooling. The pooled graph embedding is passed to a classifier head that predicts whether the graph is valid or invalid.

The node feature vector used by this vertex-GNN has ten components:

$$\phi(v) = \big[x/S,\ y/S,\ \deg(v),\ b(v),\ \mu_\alpha,\ \sigma_\alpha,\ \min\alpha,\ \max\alpha,\ K(v),\ M(v)\big],$$

where S is the coordinate scale, $b(v)$ is a boundary indicator, μ_α, σ_α, $\min\alpha$, and $\max\alpha$ summarize the angular gaps around the vertex, $K(v)$ is the local Kawasaki residual, and $M(v)$ is the local Maekawa residual.

The vertex-GNN gives the genetic algorithm a learned estimate of whether a graph resembles the training distribution of valid crease patterns. However, this score is only statistical. It is not used as a proof of flat-foldability.

6.3 Line-Graph Classifier

The second classifier is a line-graph GNN. Instead of treating crease intersections as nodes, it treats the creases themselves as nodes. Two line-graph nodes are adjacent when the corresponding creases meet at a common endpoint in the original crease pattern.

This representation gives the model a crease-level view of the pattern. The line graph includes features such as fold type, segment geometry, shared-vertex angle, cyclic adjacency, endpoint degree parity, and local Kawasaki/Maekawa residuals. This is useful because some generated patterns can look plausible at the vertex level but still be weak as crease systems.

In the final genetic algorithm, the line-GNN is used as a conservative filter. Its validity probability is converted into a penalty when the candidate falls below the chosen threshold. This pushes the search away from crease arrangements that look poor at the crease-adjacency level.

6.4 Genetic Algorithm

The generator is a genetic algorithm over embedded graph candidates. Each individual in the population is a NetworkX crease-pattern graph. The initial population is seeded from random planar graphs inside the square boundary. Candidates are then evolved using mutation, selection, elitism, shared fitness, diversity injection, and repeated repair.

The main mutations include:

- flipping a mountain fold to a valley fold, or vice versa;
- moving a mutable interior vertex;
- adding a short non-crossing crease;
- removing an interior crease when the graph is sufficiently dense;
- applying local Kawasaki repair after geometry changes.

The algorithm uses elitism, so the best candidates survive to the next generation. It also uses shared fitness and diversity selection, so the population does not collapse immediately to many copies of the same graph. Periodically, fresh repaired random seeds are injected to keep exploration active.

6.5 Topology Repair

Z3 can only repair Maekawa labels if the graph topology is suitable. In particular, a true interior vertex must have even non-border degree. If an interior vertex has odd non-border degree, no mountain-valley assignment can satisfy Maekawa's theorem there.

The topology repair module fixes this before calling Z3. It identifies bad interior vertices and tries to repair parity by removing safe odd-odd creases, removing odd interior-to-boundary creases when connectivity is preserved, or adding short non-crossing odd-odd creases. Boundary vertices are excluded from Maekawa constraints because the theorem applies to interior crease vertices.

6.6 Z3 Maekawa Repair

The Maekawa repair is formulated as a Boolean optimization problem. Each non-border crease becomes a Boolean variable:

$$x_e = \begin{cases} 1, & \text{if crease } e \text{ is a mountain fold,} \\ 0, & \text{if crease } e \text{ is a valley fold.} \end{cases}$$

For each interior vertex of non-border degree d, Maekawa's theorem requires the number of mountain folds to be either

$$\frac{d}{2} - 1 \quad \text{or} \quad \frac{d}{2} + 1.$$

The Z3 solver searches for a fold assignment satisfying these constraints while minimizing the number of changed crease labels. This makes the repair conservative: it changes only what is needed to satisfy Maekawa. If the topology is still incompatible, the candidate receives a topology or Z3 failure penalty.

6.7 Kawasaki Projection and Gradient Geometry Repair

Kawasaki's theorem is geometric. It depends on the sector angles around each vertex, so it cannot be fixed by only changing mountain-valley labels. The final algorithm therefore includes both direct Kawasaki projection and gradient geometry repair.

The direct Kawasaki projection repair computes the angular gaps around a problematic vertex, projects those gaps toward a configuration whose alternating sector-angle sums match the Kawasaki condition, and then proposes small coordinate movements to realize the improved angles. Proposed changes are rejected if they create crossings or fail to improve the local objective.

The gradient geometry repair uses PyTorch to optimize mutable node coordinates. Its loss includes Kawasaki residual, optional symmetry error, short-edge penalties, angle-gap penalties, and a small movement penalty. Boundary nodes are fixed throughout the repair. This is important because otherwise the square sheet itself could deform, which is not allowed.

6.8 Symmetry Handling

Symmetry is treated as a generation constraint rather than only as a visual penalty. The final generator allows symmetry to be turned on or off. If symmetry is enabled, the user can choose vertical symmetry or diagonal symmetry.

When symmetry is active, mutation is performed on a half-graph. The half-graph is then reflected to reconstruct the full crease pattern. This is more reliable than simply nudging pairs of mirror nodes, because it prevents the two halves from slowly drifting apart. After reflection, the square boundary is rebuilt, crossing edges are removed, features are recomputed, and the graph is relabeled. This keeps the final drawing inside a clean square boundary.

6.9 Fitness Function

The fitness function combines learned validity, mathematical constraints, and design preferences. A simplified version of the final scoring rule is

$$F(G) = P_{\mathrm{GNN}}(G) + 0.20\,C(G) - w_K K(G) - 0.25\,M(G) - w_S S(G) \\ - 0.30\,N(G) - L_{\mathrm{line}}(G) - P_{\mathrm{Z3}}(G) - P_{\mathrm{topo}}(G).$$

Here P_{GNN} is the vertex-GNN validity score, $C(G)$ rewards structural complexity, $K(G)$ is the Kawasaki penalty, $M(G)$ is the Maekawa penalty, $S(G)$ is the symmetry penalty, $N(G)$ is the novelty penalty, and $L_{\mathrm{line}}(G)$ is the line-GNN penalty computed from the line-GNN validity probability. The terms $P_{\mathrm{Z3}}(G)$ and $P_{\mathrm{topo}}(G)$ are hard penalties for failed Maekawa repair or failed topology repair.

The final algorithm also uses hard gates. Candidates with too few interior creases, edge crossings, failed topology repair, failed Z3 repair, nonzero Maekawa residual beyond tolerance, or Kawasaki residual above the current generation's threshold receive invalid fitness values. This prevents the GA from preserving candidates that score well statistically but fail the exact mathematical checks.

The Kawasaki weight increases over generations. Early in the search, the genetic algorithm is allowed to explore rough structures. Later, the fitness function becomes stricter about local flat-foldability. This schedule worked better than making Kawasaki overwhelming from the beginning, because an overly strict early penalty can destroy exploration and cause the population to become empty or overly simple.

6.10 Overall Repair Sequence

For each candidate, the final repair pipeline follows the same general order:

1. repair topology so that interior non-border degrees are compatible with Maekawa;
2. call Z3 to repair mountain-valley labels and enforce exact interior Maekawa constraints;
3. apply direct Kawasaki projection to reduce sector-angle residuals;
4. apply gradient geometry repair to further reduce Kawasaki and symmetry error;
5. reject or penalize candidates that introduce crossings or fail the theorem checks.

This sequence is important. Maekawa repair must come after topology repair, because Z3 cannot solve an impossible parity problem. Kawasaki repair must act on geometry, because fold-label changes alone cannot fix sector angles. Symmetry handling is integrated into generation and repair so that the search does not waste effort on asymmetric variants when symmetry is enabled.

7 Highlights of Coding and Implementation Details

One of the most important implementation decisions was to separate learned scoring from exact repair. The GNNs are used for plausibility, while theorem-based modules are used for correctness. This made debugging clearer because each component had a specific responsibility.

The following modules were central:

- `GNN_Classifier.py`: trains the vertex-based GIN classifier.
- `LineGraph_Classifier.py`: trains the crease-level line-GNN classifier.
- `line_graph_features.py`: converts crease-pattern graphs into line graphs.
- `topology_even_repair.py`: repairs even-degree topology before Z3.
- `maekawa_z3_repair.py`: solves Maekawa fold assignment using Z3.
- `kawasaki_projection_repair.py`: directly repairs Kawasaki sector-angle residuals.
- `gradient_geometry_repair.py`: performs coordinate optimization for Kawasaki and symmetry.
- `Genetic_Algo_Z3_Topology_Symmetry_Kawasaki.py`: combines the latest generation pipeline.
- `cp_io.py`: exports generated candidates back to editable `.cp` files.

The code uses several data structures repeatedly. NetworkX graphs store the crease patterns. PyTorch Geometric `Data` objects store graph tensors for GNN training. Shapely line segments and spatial indexes are used for crossing checks. Z3 Boolean variables represent mountain-valley assignments. Pickle files store graph datasets and generated candidates.

A small but important detail was the treatment of boundary vertices. The boundary of the paper is part of the graph for visualization and export, but it should not be treated the same as an interior flat-foldability vertex. Maekawa and Kawasaki checks are therefore focused on true interior vertices. This distinction fixed several misleading diagnostics where boundary behavior was being mixed with interior foldability constraints.

Another important detail was exact export. Generated candidates are saved as NetworkX graphs for analysis and also exported as `.cp` files so they can be opened and edited in external crease-pattern tools. This made it possible to manually inspect outputs and compare automatic repairs with hand repairs.

8 Case Studies and Results

8.1 Baseline Genetic Algorithm

The original genetic algorithm could generate visually interesting crease-pattern graphs, but it struggled with local flat-foldability. Early outputs often had high Kawasaki residuals, nonzero Maekawa residuals, odd-degree interior vertices, and low line-GNN scores. In some cases, the generator improved the GNN score while making the mathematical constraints worse.

This showed that the learned classifier alone was not enough. A graph can resemble a crease pattern statistically and still fail the exact local theorems. The baseline GA therefore motivated the later repair-based pipeline.

8.2 Line-GNN Filtering

The line-GNN achieved strong held-out performance. In one test run, the full line-GNN test accuracy was approximately 97.62%, with confusion matrix

$$\begin{bmatrix} 66 & 0 \\ 3 & 57 \end{bmatrix},$$

where rows are true labels and columns are predicted labels. It also classified most real crease patterns as valid and most corrupted patterns as invalid.

When added to the GA, the line-GNN acted as a conservative filter. It helped push the generator away from graph structures that looked poor at the crease level. However, line-GNN filtering alone did not solve the problem. Early line-filtered generations still produced candidates with high Kawasaki and Maekawa violations.

8.3 Z3 and Topology Repair

The Z3 Maekawa solver worked well when the graph topology was suitable. It could take an already generated graph and change a small number of fold labels so that the interior Maekawa residual became zero. However, many generated graphs had odd non-border degree at interior vertices. In those cases, no mountain-valley assignment could satisfy Maekawa, so Z3 correctly returned a topology failure.

After adding even-degree topology repair before Z3, the results improved significantly. By generation 20 in one topology-repaired experiment, 53 out of 54 repair attempts were Z3 satisfiable. The best generated candidate from this stage had line-GNN valid probability

about 0.9246, and all six diverse top candidates were predicted valid by the line-GNN. The exact interior Maekawa residual was 0.

This was an important turning point in the project. It showed that the Maekawa problem should not be treated only as a label problem. The topology must first be made compatible with Maekawa.

8.4 Kawasaki-Focused Repair

Kawasaki proved to be the hardest constraint. Unlike Maekawa, it cannot be solved by assigning labels. It requires changing the geometry of the embedded graph. The project therefore added direct Kawasaki projection and gradient geometry repair.

A vertical-symmetry run, whose top-six plot was saved as `z3_symmetry_kawasaki_ga_top6`, ended with both the node-GNN and line-GNN scores high while driving Kawasaki close to zero:

Generation	GNN	Kawasaki	Maekawa	Sym. Penalty	Line-GNN	Z3 Status
1	0.000	1.347	0.000	1.000	0.000	SAT
5	0.572	0.662	0.000	0.000	0.034	SAT
10	0.519	0.226	0.000	0.000	0.950	SAT
15	0.769	0.057	0.000	0.000	0.979	SAT
20	0.884	0.052	0.000	0.000	0.985	SAT
25	0.910	0.037	0.000	0.000	0.983	SAT
30	0.902	0.027	0.000	0.000	0.985	SAT

The symmetry column is a penalty, so 0.000 means no measured symmetry violation. This happens because the symmetric version mutates one half of the page and rebuilds the full crease pattern by reflection. The final diverse top-six candidates were also all Z3-satisfiable with exact Maekawa residual zero:

Rank	Fitness	GNN	Kawasaki	Maekawa	Z3 Status
1	1.044	0.909	0.025	0.000	SAT
2	1.035	0.911	0.032	0.000	SAT
3	1.038	0.911	0.032	0.000	SAT
4	1.030	0.914	0.037	0.000	SAT
5	1.045	0.911	0.031	0.000	SAT
6	1.028	0.902	0.027	0.000	SAT

Across the run, the best-ever fitness reached 1.0558. The final diverse top-six candidates had fitness values from 1.028 to 1.045. This showed that the explicit repair pipeline can

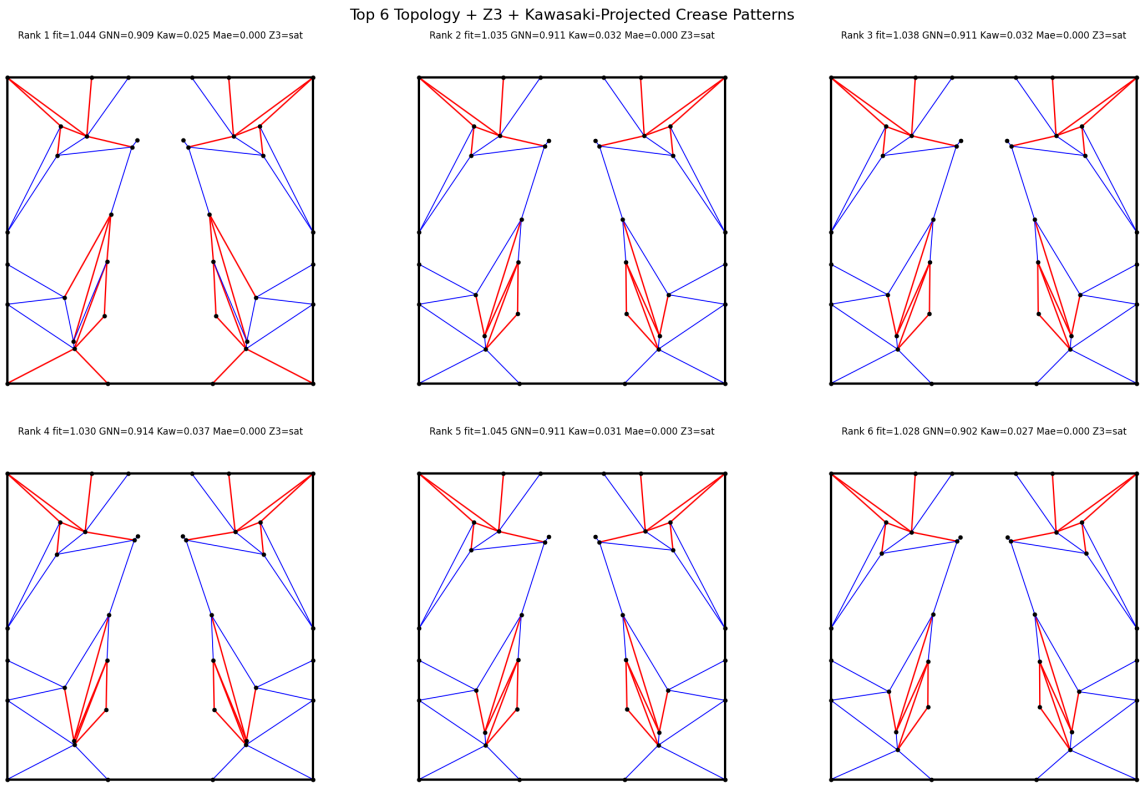

Figure 4: Top-six crease patterns from the symmetry-aware Z3 and Kawasaki-projected GA run.

push Kawasaki close to zero while keeping Maekawa exactly zero. The symmetry-enforced run also preserved high classifier scores: by generation 30, the node-GNN score was 0.902 and the line-GNN score was 0.985. This suggests that adding symmetry repair and direct Kawasaki projection made the generated theorem-satisfying patterns much more compatible with the learned validity models.

8.5 Symmetry-Constrained Generation

Symmetry was added to improve visual structure and reduce the search space. The final version supports vertical symmetry, diagonal symmetry, or no enforced symmetry. The most reliable method was not to repair symmetry after the fact, but to mutate a half-graph and reflect it exactly. This keeps the output symmetric by construction.

Symmetry also made some repairs more stable. When both halves of the crease pattern are generated together, the system avoids the common failure mode where one side improves while the mirrored side drifts away.

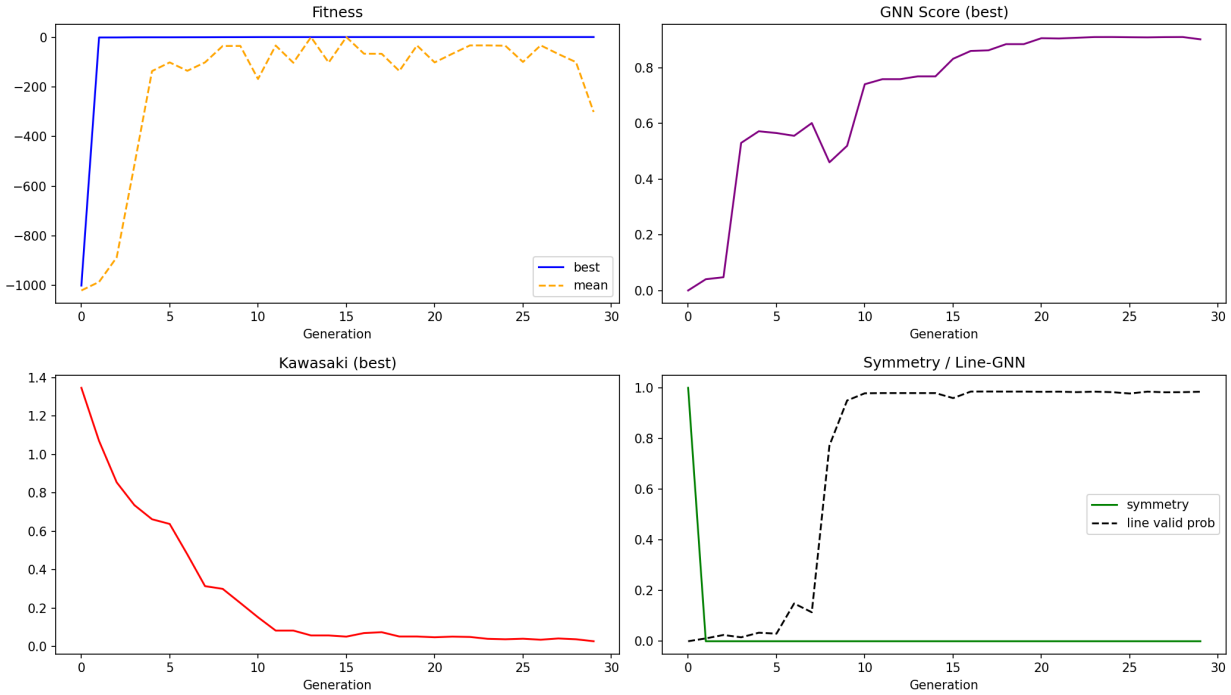

Figure 5: Convergence plot from the above run.

8.6 Final Results:

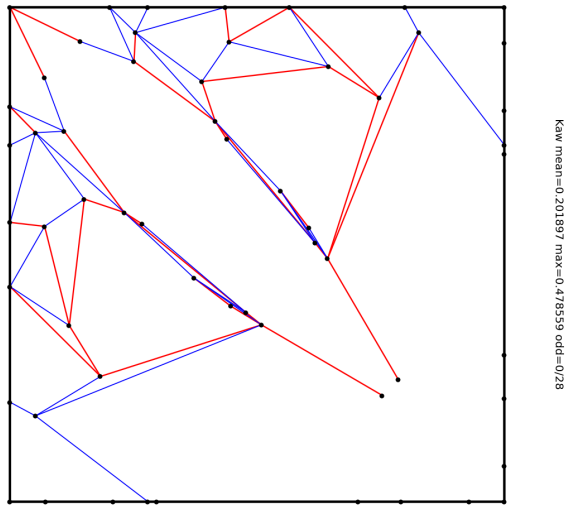

Figure 6: Generated Crease Pattern with Diagonal Symmetry

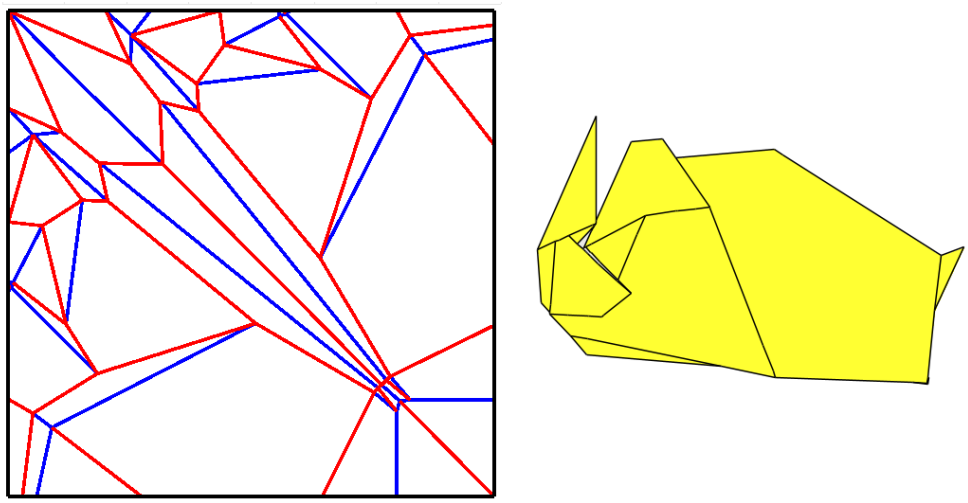

Figure 7: Origami Rabbit

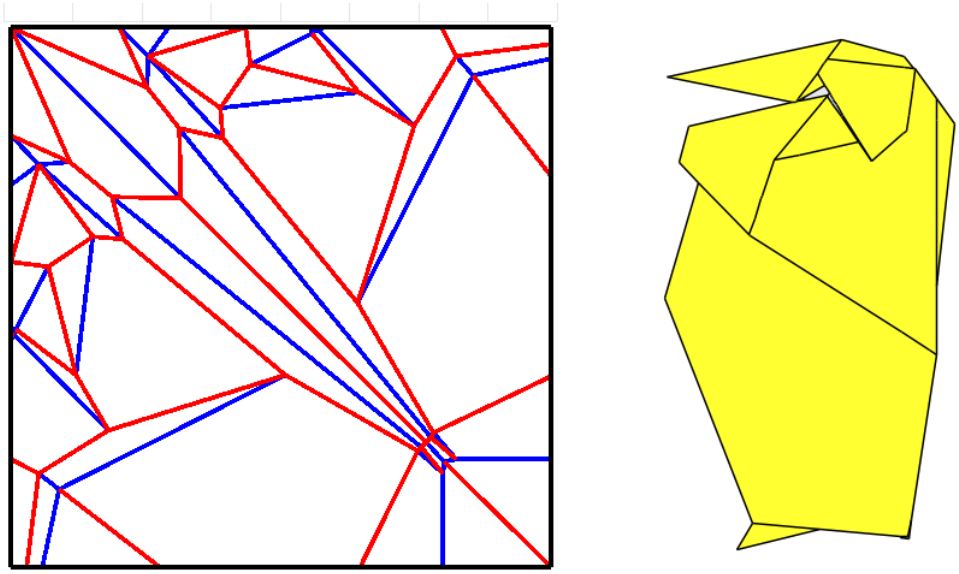

Figure 8: Origami Penguin

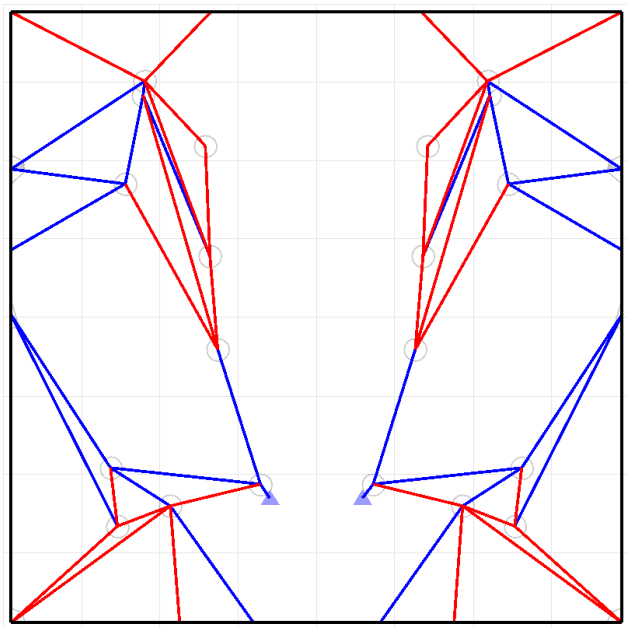

Figure 9: Generated Crease Pattern with Vertical Symmetry

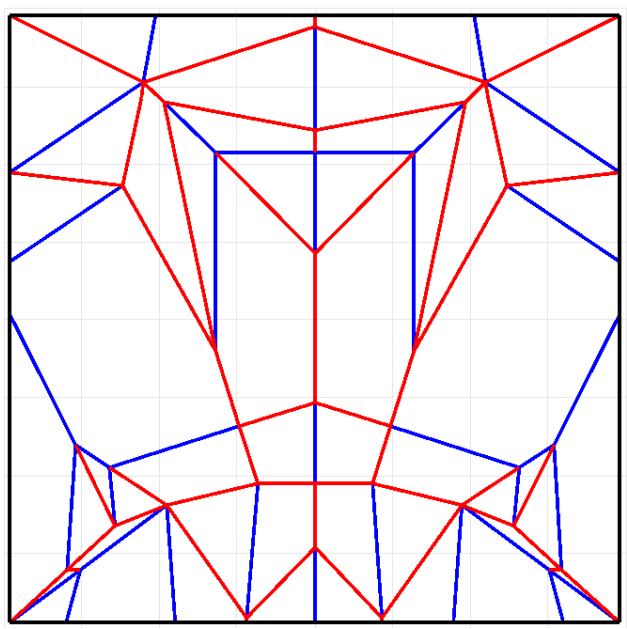

Figure 10: Fixed crease pattern

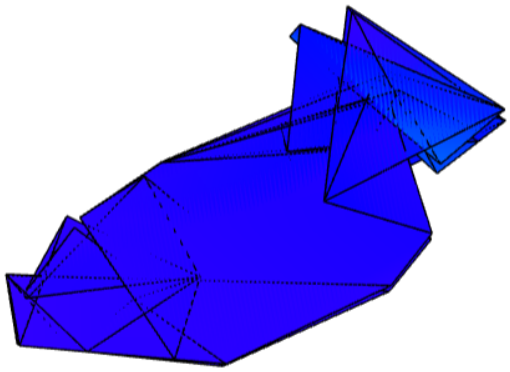

Figure 11: Origami Rabbit

9 Bonus: Dashboard, Dataset, and Reusable Code

As a bonus component, we built a reusable Streamlit dashboard for running and inspecting the origami crease-pattern generator. The public dashboard can be accessed here: `Origami Generator Dashboard`. The full project repository is available here: `GitHub Repository`.

The dashboard provides an interface around the final Z3, topology, symmetry, and Kawasaki-repaired generator. Instead of requiring the user to run the Python scripts manually, the dashboard exposes controls for population size, number of generations, elite count, mutation count, and symmetry mode. After a run, it displays the generated top candidates, reports constraint diagnostics, and provides downloadable crease-pattern files.

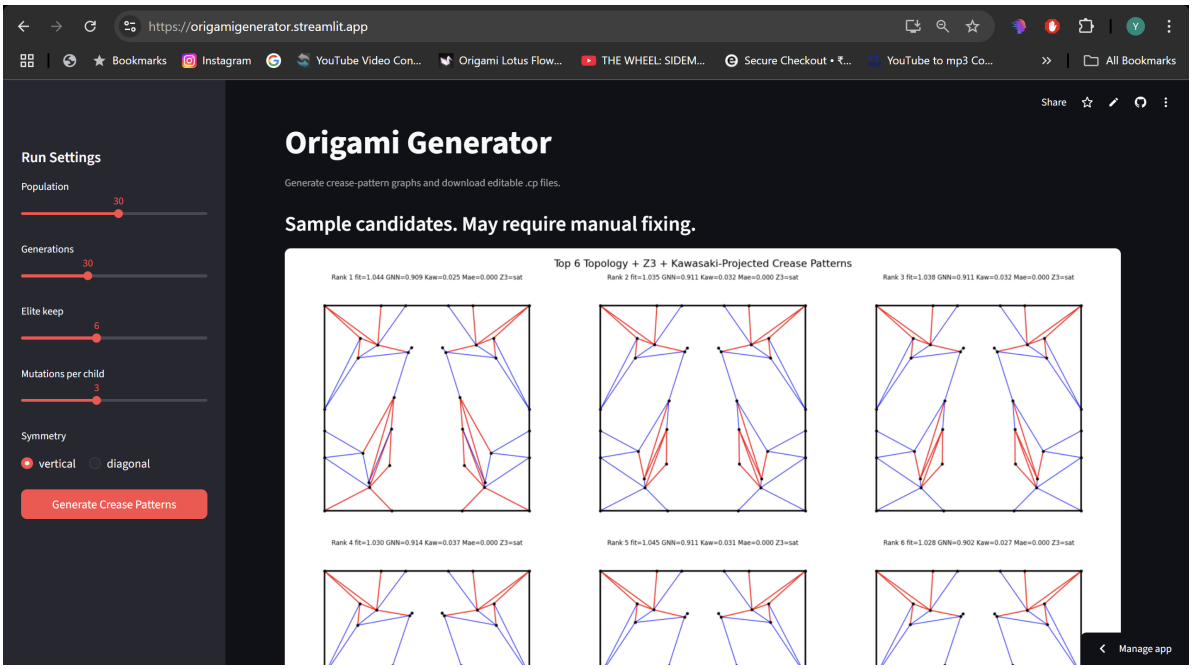

Figure 12: Streamlit dashboard for generating, viewing, and downloading crease-pattern candidates.

In addition to the dashboard, the project repository contains reusable code for the full generation algorithm. This includes code for graph parsing, graph neural network classification, line-graph filtering, topology repair, Z3-based Maekawa repair, Kawasaki projection repair, gradient geometry repair, visualization, and CP-file export. This makes the project easier to extend because future experiments can reuse the same pipeline instead of rebuilding each component from scratch.

We also make the dataset available as a reusable Python pickle file. The main dataset contains 626 real crease-pattern graphs parsed from `.cp` files and stored as:

`graphs.pkl`

Each graph in this pickle file is stored as a NetworkX graph, where nodes contain geometric and local structural features, and edges contain fold-type labels. This dataset is used for training the GNN validity classifier and can also be reused for later experiments on origami graph learning or crease-pattern generation.

The generated CP-file dataset is stored in:

`z3_symmetry_kawasaki_diverse_top6_cp/`

This folder contains six diverse top-ranked generated crease patterns:

`rank_1.cp`, `rank_2.cp`, ..., `rank_6.cp`

The best overall candidate is exported separately as:

`z3_symmetry_kawasaki_best_generated.cp`

Each `.cp` file is a plain-text crease-pattern file. Every row stores one crease segment in the format:

```
fold_type x1 y1 x2 y2
```

Here, `fold_type` is `1` for a border edge, `2` for a mountain fold, and `3` for a valley fold. The four coordinates give the two endpoints of the crease segment. Because the files are exported in an editable CP format, they can be opened in external crease-pattern tools, manually adjusted, or reused as generated examples in later experiments.

Overall, this bonus component turns the project from a collection of scripts into a reusable tool and dataset. The dashboard provides an accessible interface for running the generator, the GitHub repository preserves the implementation, and the pickle/CP-file outputs make the data reusable for future origami generation and graph-learning experiments.

10 Conclusions and Possible Future Directions

This project built a working hybrid pipeline for generating origami crease-pattern candidates. Crease patterns are represented as embedded graphs, classified using both vertex-GNN and line-GNN models, evolved using a genetic algorithm, and repaired using topology, Z3, Kawasaki projection, gradient geometry, and symmetry constraints.

The most successful part of the system is the combination of topology repair and Z3. Once interior non-border degrees are made compatible with Maekawa's theorem, Z3 can assign mountain-valley labels with exact interior Maekawa residual 0. The Kawasaki repair modules also substantially reduce geometric error, in some experiments reaching residuals close to zero.

The project does not yet solve full origami design, but it does move beyond purely visual generation. The generated candidates are not just drawings; they are graph objects with explicit fold labels, editable `.cp` exports, learned validity scores, and local flat-foldability diagnostics.

Several future directions are natural:

- **Retrain the GNNs with repaired outputs.** This may reduce the conflict between low Kawasaki residual and low GNN confidence.

- **Add Big-Little-Big repair.** Maekawa and Kawasaki are currently handled more directly than Big-Little-Big.
- **Use a global Kawasaki optimizer.** A larger constrained optimization problem may work better than repeated local repairs.
- **Add face-based constraints.** Two-colorability and dual-graph checks could help move from local vertex validity toward global crease-pattern validity.
- **Improve topology-aware mutation.** Instead of repairing odd-degree vertices after mutation, mutations could be designed to preserve even non-border degree automatically.
- **Expand the dataset.** More real crease patterns would make the classifiers less brittle and improve novelty evaluation.

The broader conclusion is that origami crease-pattern generation benefits from combining machine learning with classical mathematical constraints. The graph neural networks provide a learned sense of what real crease patterns look like, while the repair modules enforce the theorems that make those patterns meaningful as origami. This combination gives a practical path toward generating novel, editable, and locally foldable crease-pattern candidates.

References

[1] Hugo A. Akitaya, Erik D. Demaine, and Jason S. Ku. Computing flat-folded states. 2023.

[2] Marshall Bern and Barry Hayes. The complexity of flat origami. In *Proceedings of the 7th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 175–183, 1996.

[3] Erik D. Demaine and Joseph O'Rourke. *Geometric Folding Algorithms: Linkages, Origami, Polyhedra*. Cambridge University Press, 2007.

[4] *Origami^3: Third International Meeting of Origami Science, Mathematics, and Education*. A K Peters, Ltd., 2002.

[5] Thomas Hull. On the mathematics of flat origamis. 2002.

[6] Maya Kraft and Jason Ku. Rabbit Ear: A JavaScript origami software library. 2023.

[7] Gerben Oolbekkink, Silas Reschke (undetrox), Mu-Tsun Tsai, and contributors. Oriedita: Origami crease pattern editor and folding simulator. https://github.com/oriedita/oriedita, 2025.

[8] Leland Schipper. Mathematical origami. *Central College Synaptic Writing Anthology*, May 2019.

[9] First AI Generated Origami (That I Know Of). YouTube video. https://www.youtube.com/watch?v=dPPg-s4lK6Q. Accessed May 3, 2026.